Jr DevOps Engineer — Practical Assessment

Important Instructions:

- This is a **hands-on, build-it-yourself** assessment. You must actually create working infrastructure, write real code, and submit proof.
- You may use AI tools, documentation, and the internet.
- All work must be submitted in a **single public GitHub repository**.
- Budget: Use **free tiers only**. No paid resources required.

Allowed Platforms (All Free Tier)

Service	Free Option
Cloud VM	Oracle Cloud Free Tier (2 AMD VMs + 1 ARM VM, always free) OR AWS Free Tier (1x t2.micro) OR local VM (VirtualBox / Multipass)
Container Registry	GitHub Container Registry (`ghcr.io`) — free for public repos
CI/CD	GitHub Actions — 2,000 free minutes/month for private repos, unlimited for public
DNS	DuckDNS (free dynamic DNS) OR nip.io / sslip.io (no signup needed) OR Cloudflare Free
TLS/SSL	Let's Encrypt via Caddy (automatic) or Certbot. For local VMs, use `mkcert`
Monitoring	Uptime Kuma (self-hosted, open source)
Secrets	GitHub Actions Secrets + `.env` file (excluded from Git)
AWS Simulation	**LocalStack (free hobby tier) — optional, for Terraform practice**

> **No cloud VM?** Use a local machine or VirtualBox VM. Provide proof via screen recordings and `curl` output. For TLS, use `mkcert` for local certificates. You will not be penalized.

The Mission

You are setting up the infrastructure for **StatusPulse** — a lightweight status page and health monitoring API. The application code is provided below. **Your job is not to write the application — it's to build everything around it so it runs reliably in production.**

You will: 1. Containerize it properly 2. Set up a CI/CD pipeline 3. Deploy it to a server with HTTPS 4. Add monitoring and alerting 5. Write Infrastructure as Code 6. Harden security

---

The Starter Application

Create these files in your repo under `app/`. This is the application you will deploy — **do not modify the application code**.

`app/main.py`

```
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel
from datetime import datetime, timezone
import os
import redis
import psycopg2
import json

app = FastAPI(title="StatusPulse", version="1.0.0")


def get_db_connection():
    return psycopg2.connect(
        host=os.environ["DB_HOST"],
        port=os.environ.get("DB_PORT", "5432"),
        dbname=os.environ["DB_NAME"],
        user=os.environ["DB_USER"],
        password=os.environ["DB_PASSWORD"],
    )


def get_redis_connection():
    return redis.Redis(
        host=os.environ.get("REDIS_HOST", "redis"),
        port=int(os.environ.get("REDIS_PORT", "6379")),
        password=os.environ.get("REDIS_PASSWORD", None),
        decode_responses=True,
    )


def init_db():
    conn = get_db_connection()
    cur = conn.cursor()
    cur.execute("""
        CREATE TABLE IF NOT EXISTS services (
            id SERIAL PRIMARY KEY,
            name VARCHAR(100) UNIQUE NOT NULL,
            url VARCHAR(500) NOT NULL,
            status VARCHAR(20) DEFAULT 'unknown',
            last_checked TIMESTAMP,
            response_time_ms INTEGER
        )
    """)
    cur.execute("""
        CREATE TABLE IF NOT EXISTS incidents (
            id SERIAL PRIMARY KEY,
            service_name VARCHAR(100) NOT NULL,
            title VARCHAR(200) NOT NULL,
            description TEXT,
            severity VARCHAR(20) DEFAULT 'minor',
            status VARCHAR(20) DEFAULT 'investigating',
            created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
```

```
            resolved_at TIMESTAMP
        )
    """)
    conn.commit()
    cur.close()
    conn.close()


@app.on_event("startup")
async def startup():
    init_db()


@app.get("/health")
def health_check():
    checks = {"api": "healthy", "database": "unknown", "redis": "unknown"}
    try:
        conn = get_db_connection()
        cur = conn.cursor()
        cur.execute("SELECT 1")
        cur.close()
        conn.close()
        checks["database"] = "healthy"
    except Exception as e:
        checks["database"] = f"unhealthy: {str(e)}"
    try:
        r = get_redis_connection()
        r.ping()
        checks["redis"] = "healthy"
    except Exception as e:
        checks["redis"] = f"unhealthy: {str(e)}"
    overall = (
        "healthy"
        if all(v == "healthy" for v in checks.values())
        else "degraded"
    )
    return {
        "status": overall,
        "checks": checks,
        "timestamp": datetime.now(timezone.utc).isoformat(),
    }


class ServiceCreate(BaseModel):
    name: str
    url: str


@app.post("/services")
def add_service(service: ServiceCreate):
    conn = get_db_connection()
    cur = conn.cursor()
```

```
    try:
        cur.execute(
            "INSERT INTO services (name, url) VALUES (%s, %s) RETURNING id",
            (service.name, service.url),
        )
        service_id = cur.fetchone()[0]
        conn.commit()
        return {"id": service_id, "name": service.name, "url": service.url}
    except psycopg2.errors.UniqueViolation:
        conn.rollback()
        raise HTTPException(status_code=409, detail="Service already
exists")
    finally:
        cur.close()
        conn.close()


@app.get("/services")
def list_services():
    conn = get_db_connection()
    cur = conn.cursor()
    cur.execute(
        "SELECT id, name, url, status, last_checked, response_time_ms "
        "FROM services"
    )
    rows = cur.fetchall()
    cur.close()
    conn.close()
    return [
        {
            "id": r[0], "name": r[1], "url": r[2],
            "status": r[3], "last_checked": str(r[4]),
            "response_time_ms": r[5],
        }
        for r in rows
    ]


class IncidentCreate(BaseModel):
    service_name: str
    title: str
    description: str = ""
    severity: str = "minor"


@app.post("/incidents")
def create_incident(incident: IncidentCreate):
    conn = get_db_connection()
    cur = conn.cursor()
    cur.execute(
        "INSERT INTO incidents (service_name, title, description, severity)
"
```

```
        "VALUES (%s, %s, %s, %s) RETURNING id",
        (incident.service_name, incident.title,
         incident.description, incident.severity),
    )
    incident_id = cur.fetchone()[0]
    conn.commit()
    cur.close()
    conn.close()
    try:
        r = get_redis_connection()
        r.publish(
            "incidents",
            json.dumps({
                "id": incident_id,
                "title": incident.title,
                "severity": incident.severity,
            }),
        )
    except Exception:
        pass
    return {"id": incident_id, "status": "investigating"}


@app.get("/incidents")
def list_incidents():
    conn = get_db_connection()
    cur = conn.cursor()
    cur.execute(
        "SELECT id, service_name, title, severity, status, "
        "created_at, resolved_at FROM incidents ORDER BY created_at DESC"
    )
    rows = cur.fetchall()
    cur.close()
    conn.close()
    return [
        {
            "id": r[0], "service_name": r[1], "title": r[2],
            "severity": r[3], "status": r[4],
            "created_at": str(r[5]), "resolved_at": str(r[6]),
        }
        for r in rows
    ]


@app.get("/")
def root():
    return {
        "service": "StatusPulse",
        "version": "1.0.0",
        "docs": "/docs",
        "health": "/health",
    }
```

`app/requirements.txt`

```
fastapi==0.104.1
uvicorn==0.24.0
gunicorn==21.2.0
psycopg2-binary==2.9.9
redis==5.0.1
pydantic==2.5.2
```

Task 1 — Dockerize the Application (30 Marks)

(a) Production Dockerfile (10 Marks)

Write a Dockerfile that: - Uses multi-stage build (builder + slim runtime) - Runs as a non-root user - Optimizes layer caching (install dependencies before copying app code) - Includes a `HEALTHCHECK` instruction - Final image must be **under 200MB**

(b) Docker Compose for local development (10 Marks)

Write a `docker-compose.yml` that runs: app + PostgreSQL + Redis. It must: - Load environment variables from a `.env` file (provide `.env.example` with placeholders) - Include health checks for all three services - Include memory limits for each service - Use named volumes for PostgreSQL persistence - Use a custom network (not default bridge)

(c) `.dockerignore` file (2 Marks)

(d) `Makefile` with these targets (8 Marks): - `make build` — build the Docker image - `make up` — start all services - `make down` — stop all services - `make logs` — tail logs - `make test` — health check the running service via `curl` - `make clean` — remove containers, images, and volumes - `make shell` — open bash inside the running app container

Proof Required

- Screenshot: `docker images` — your image must be under 200MB
- Screenshot: `docker compose ps` — all 3 services healthy
- Screenshot: `curl localhost:<port>/health` — all checks healthy
- Terminal output: `make test` succeeding

Task 2 — CI/CD Pipeline (35 Marks)

(a) CI workflow — `.github/workflows/ci.yml` (12 Marks)

Runs on every push and PR to `main`: - Lint Python code with `ruff` - Scan Dockerfile with `hadolint` - Build Docker image - Start full stack via Docker Compose inside the runner - Run

integration tests against the live stack (hit all API endpoints, verify status codes and JSON response shapes) - Tear down the stack - Upload test results as a GitHub Actions artifact

(b) Deploy workflow — `.github/workflows/deploy.yml` (15 Marks)

Runs on push to `main` only (after CI passes): - Build and tag the image with commit SHA and `latest` - Push to GitHub Container Registry (`ghcr.io`) - SSH into your server and deploy the new image - Run a post-deployment health check (`/health` endpoint) - If health check fails, **automatically rollback** to the previous image - Send a notification on success/failure (GitHub Issue comment, Slack webhook, Discord webhook, or email — your choice)

(c) Integration test script — `tests/test_integration.sh` (8 Marks)

- Test all endpoints: `GET /health`, `POST /services`, `GET /services`, `POST /incidents`, `GET /incidents`
- Verify HTTP status codes (200, 201, 409 for duplicate)
- Verify JSON response structure
- Exit non-zero on any failure
- Clear pass/fail output per test

Proof Required

- Link to GitHub repo with workflow files
- Link to at least **3 successful CI runs** (green checks)
- Link to a **successful deploy run** showing image push + deploy + health check
- Link to a **failed CI run** (intentionally break something to prove it catches errors)
- Screenshot: GitHub Container Registry with SHA-tagged images
- Screenshot: deployment notification received

Task 3 — Deploy to a Live Server (35 Marks)

(a) Server hardening (10 Marks)

Provision a server (free VM or local) and configure: - SSH hardened: disable root login, disable password auth, change default port - Firewall (UFW or iptables): allow only custom SSH port, 80, and 443 - Non-root deploy user with Docker permissions - Automatic security updates (`unattended-upgrades`) - Swap space configured (for low-RAM free tier VMs)

(b) Deploy with HTTPS (15 Marks)

- StatusPulse + PostgreSQL + Redis running via Docker Compose
- **Caddy** or **Nginx + Certbot** as reverse proxy with automatic TLS
- Accessible at `https://<your-domain>/` with valid certificate
- HTTP redirects to HTTPS
- `/health` returns all-healthy
- `/docs` Swagger UI is accessible

(c) Deployment script — `scripts/deploy.sh` (10 Marks)

A script that runs on the server to: - Pull latest image from `ghcr.io` - Zero-downtime deploy: start new → health check → stop old - Rollback automatically if health check fails - Log all actions with timestamps - Idempotent (safe to run repeatedly)

Proof Required

- Your live URL — we will visit `https://<your-domain>/health`
- Screenshot: `https://<your-domain>/docs` showing Swagger UI over HTTPS
- Screenshot: `curl -vI https://<your-domain>/` showing valid TLS certificate
- Screenshot: `sudo ufw status` showing firewall rules
- Screenshot: `sshd_config` showing hardened SSH settings
- Terminal output: `deploy.sh` running a successful deployment
- Terminal output: `deploy.sh` rolling back after deploying a deliberately broken image

---

Task 4 — Monitoring & Alerting (30 Marks)

(a) Deploy Uptime Kuma (10 Marks)

- Running via Docker on your server
- Accessible at `https://<your-domain>/status` or `https://status.<your-domain>/`
- Monitors configured:
- StatusPulse `/health` endpoint (every 60s)
- PostgreSQL (TCP port check)
- Redis (TCP port check)
- TLS certificate expiry
- Public status page enabled

(b) Alerting — configure at least 2 notification channels (10 Marks)

Choose any two: Email, Slack, Discord, Telegram, or Ntfy.sh (all free).

Demonstrate alerts by: 1. Stop the StatusPulse container 2. Show the alert received 3. Restart the container 4. Show the recovery notification

(c) Health monitor script — `scripts/health-monitor.sh` (10 Marks)

A cron job that runs every 5 minutes: - Check `/health` endpoint returns HTTP 200 with valid JSON - Check disk usage > 80% → alert - Check memory usage > 90% → alert - Check all expected Docker containers are running - Check TLS certificate expires within 14 days → alert - Send alerts via webhook (`$ALERT_WEBHOOK_URL` env var) - Log to `/var/log/statuspulse-monitor.log` with timestamps - Handle failures gracefully (curl timeout, missing commands)

Proof Required

- Public Uptime Kuma status page URL — we will visit it
- Screenshot: Uptime Kuma dashboard showing all 4 monitors green
- Screenshots: Alert notifications on both channels (down + recovery)
- Screenshot: `crontab -l` showing the cron job
- Show `/var/log/statuspulse-monitor.log` with at least 1 hour of entries
- Demonstrate disk alert: `fallocate -l 5G /tmp/diskfill && sleep 310 && rm /tmp/diskfill`

Task 5 — Infrastructure as Code (30 Marks)

(a) Terraform or Ansible — your choice (20 Marks)

Write IaC that can recreate your entire setup from scratch.

If Terraform: - Provider config (AWS, OCI, Hetzner, or LocalStack for simulation) - VM instance, security group / firewall, DNS record (if applicable) - Output the server IP - Use variables for all configurable values - Include `variables.tf` with descriptions and defaults

If Ansible: - Inventory file + playbook that configures a fresh Ubuntu VM into your complete setup: - Install Docker + Compose - Configure firewall + harden SSH + set up swap - Deploy StatusPulse stack + reverse proxy with TLS - Deploy Uptime Kuma - Install cron jobs - Must be idempotent (run twice → no changes on second run)

In either case: include a `README.md` with step-by-step usage instructions.

(b) Backup script — `scripts/backup.sh` (10 Marks)

- Dump PostgreSQL to compressed file: `statuspulse_db_YYYY-MM-DD_HHMMSS.sql.gz`
- Keep only last 7 backups (rotate older ones)
- Optionally upload to S3 / object storage (if `$S3_BUCKET` is set)
- Log all actions
- Scheduled as daily cron job

Proof Required

- Complete Terraform/Ansible code in the repo
- `README.md` with clear setup instructions
- Screenshot: running the IaC tool successfully
- If Ansible: run twice, show `changed=0` on second run (idempotency proof)
- Backup script output: successful dump + rotation
- Restore the backup to a fresh database and show data is intact
- `crontab -l` showing daily backup schedule

Task 6 — Security Hardening (20 Marks)

(a) Container image scanning (8 Marks)

- Run `trivy image <your-image>` or `docker scout` on your built image
- Fix or mitigate any HIGH / CRITICAL vulnerabilities
- Show before-and-after scan results
- Document findings and fixes in `SECURITY.md`

(b) Secret management (7 Marks)

- Zero secrets in any committed file (no passwords in Compose, Dockerfile, or code)
- `.env` file in `.gitignore`
- GitHub Actions secrets for CI/CD variables
- Document your approach in `SECURITY.md`

(c) Reverse proxy security headers + rate limiting (5 Marks)

- Rate limiting: 100 requests/minute per IP
- Headers: `X-Content-Type-Options`, `X-Frame-Options`, `Strict-Transport-Security`, `X-XSS-Protection`

Proof Required

- Trivy/Scout scan: before (with vulns) and after (fixed)
- `SECURITY.md` in repo
- `git log` showing no secrets were ever committed
- `curl -I https://<your-domain>/` showing all security headers
- Rate limit demo: `for i in $(seq 1 120); do curl -s -o /dev/null -w "%{http_code}\n" https://<your-domain>/health; done` — show 429 responses

Task 7 — Documentation (20 Marks)

(a) `README.md` in repo root (20 Marks)

Must include: - Architecture diagram (ASCII, Mermaid, or image) - Prerequisites - How to run locally with Docker Compose - How to deploy to production - How the CI/CD pipeline works - How monitoring and alerting is configured - How to backup and restore - Troubleshooting section

---

Repository Structure

Your final repo should look like:

```
statuspulse/
├── .github/
│   └── workflows/
│       ├── ci.yml
│       └── deploy.yml
├── app/
│   ├── main.py
│   └── requirements.txt
├── caddy/  (or nginx/)
│   └── Caddyfile  (or nginx.conf)
├── scripts/
│   ├── deploy.sh
│   ├── backup.sh
│   └── health-monitor.sh
├── terraform/  (or ansible/)
│   ├── main.tf  (or playbook.yml)
│   ├── variables.tf  (or vars.yml)
│   └── README.md
├── tests/
│   └── test_integration.sh
├── .dockerignore
├── .env.example
├── .gitignore
├── docker-compose.yml
├── Dockerfile
├── Makefile
├── README.md
└── SECURITY.md
```

Evaluation Criteria

Task	Topic	Marks
1	Dockerize the Application	30
2	CI/CD Pipeline	35
3	Deploy to a Live Server	35
4	Monitoring & Alerting	30
5	Infrastructure as Code	30
6	Security Hardening	20
7	Documentation	20

Task	Topic	Marks
Total		**200**

Submission Checklist

Before submitting, verify:

- [ ] Public GitHub repository link
- [ ] Live HTTPS URL where StatusPulse is running
- [ ] Uptime Kuma public status page URL
- [ ] At least 3 successful CI/CD runs in GitHub Actions
- [ ] All proof screenshots in a `screenshots/` folder in the repo

Submit your public GitHub repo link as instructed.

Engineering Department